

对抗生成与博弈训练 实验手册

第061单元 从两步更新到一次真实坍塌

本实验的交付不是一张漂亮点图，而是一份能区分“分布覆盖”“点的有效性”“训练稳定性”的证据记录。建议先读讲义第一至六节，再运行本手册；遇到符号时，把D理解为真假来源概率，把G理解为把随机数变成二维点的函数。

一 建立环境并确认起点

在当前单元目录建立独立环境，推荐Python 3.11或3.12。requirements.txt给出制作时版本，environment.yml给出Conda方案。CPU版本PyTorch请从官方安装渠道选择适合平台的命令；本实验不需要CUDA、外部数据或账户。不要加载不明来源的pt文件，本包权重只保存参数与简单元数据。

```
python -m venv .venv
# 按操作系统激活环境，再安装 requirements.txt
python -m pip install -r requirements.txt
python -m unittest -v test_experiment.py
python -O -m unittest -v test_experiment.py
python experiment.py
python make_figures.py
```

若在已有Jupyter里运行，先打印sys.executable和torch.__version__，确认Notebook使用同一解释器。正常测试与优化模式测试应全部通过。若失败，记录完整报错，先修测试再阅读生成质量；不要为得到图片而把测试删掉。完整训练通常为分钟量级，实际取决于CPU与系统负载。

数据生成函数返回两部分： $N \times 2$ 浮点坐标和N个中心编号。GAN训练只能使用坐标。检查train有4096行、reference有4096行，八个中心半径接近2，训练与参考种子不同。打印前五五行是检查形状，不是质量评价。图中每个团的真实样本数也不会恰好相等，因为中心编号由独立随机抽样决定。

第一份记录写下解释器版本、运行命令、开始和结束时间、正常与-O测试结果。先不要修改超参数。只有基线重现完成后，再开始扩展实验，避免把环境问题与算法变化混在一起。

二 先把一轮更新解释给自己听

打开experiment.py中的discriminator_step。用自己的话解释为什么生成样本需要detach。然后查看generator_step，指出为什么只冻结D的参数，而不对整个D前向使用no_grad。请在纸上画出“损失→D输出→D输入→G参数”的箭头，标出两个阶段分别在哪一处切断或保留。

运行如下最小检查，并与test_experiment.py中的自动测试对照。先复制G/D参数作为before，执行一次D步骤，再比较G参数是否逐元素不变；检查G参数梯度仍为空。接着执行一次G步骤，确认D参数没有改变，G至少一个参数改变，且所有D参数最终恢复可训练状态。只打印loss不够，因为错误实现也可能返回一个正常数字。

```
import torch
from experiment import Generator, Discriminator
from experiment import discriminator_step, generator_step

torch.manual_seed(7)
g, d = Generator(), Discriminator()
go = torch.optim.Adam(g.parameters(), lr=0.0002)
do = torch.optim.Adam(d.parameters(), lr=0.0002)
real = torch.randn(16, 2)
z = torch.randn(16, 2)
print(discriminator_step(g, d, do, real, z))
print(generator_step(g, d, go, z))
```

纸笔任务A：真假各两个样本，真实概率0.8和0.6，生成概率0.2和0.4，按两组均值相加计算判别损失。再把所有概率改为0.5，解释为什么本实现的参考值是1.3863而不是0.6931。

纸笔任务B：令 $d = \text{Sigmoid}(s)$ ，分别求 $\log(1-d)$ 和 $-\log d$ 对 s 的导数，代入 $d=0.01$ 和 $d=0.5$ 。说明“logit上的梯度较强”为什么仍不足以保证生成器所有参数获得有效方向。答案必须提到链式法则中另外两段导数，不能只写“非饱和更好”。

提交这一部分时，附两种更新的参数变化检查和手算过程。不要只提交通过截图；把判断条件写清楚，才便于他人复核你的结论。

三 完整训练三个种子并读懂指标

执行python experiment.py会重建数据并运行种子11、23、37的基准以及种子17的压力配置。输出写入outputs；同名文件会被本地重建覆盖，若需要保留自己的实验，请用--output指定另一个目录。修改轮数会使教材中的数值失效，必须在报告中写清楚。

读取results.json，为每个基准记录覆盖数、有效比例、最大条件模式占比，以及最近中心平均距离。制作时三组有效比例约为0.6243、0.6748、0.6042，三组覆盖均为8。不同软件或硬件可有小幅浮点差异；重点是先核对配置、数据和指标定义，再判断差异是否实质。

```
import json
from pathlib import Path
r = json.loads(Path('outputs/results.json').read_text())
for run in r['runs']:
    f = run['final']
    print(run['seed'], f['coverage'], f['valid_fraction'])
```

指标任务C：构造4096个点，其中每个中心放41个，剩下点放原点。调用mode_metrics，解释为什么覆盖可能达到8，而有效比例只有约8%。再构造所有点都在一个中心的常数集合，解释为什么质量很好但覆盖很差。这两组都是人为测试数据，报告中必须标注“评价器负对照”。

不要把mode_metrics内部的最近中心编号误当GAN输入标签。中心信息只用于评价。若把评价半径0.35改为0.2，覆盖与有效比例会怎样变化？先在已有保存样本上重算，不要重新训练，以隔离“评价口径改变”和“模型改变”。阈值更严后，指标下降不一定意味着模型真的退步。

看figure02和figure03时，所有子图保持同一坐标范围。观察基准是否有团间桥状点、某些团是否偏瘦、概率质量是否均匀。写三句有证据的描述，至少一句指出基准的不足。不能因为三次都8/8覆盖，就写“GAN已经完全学会真实分布”。

四 复核真正发生的坍塌

打开stress记录的diagnostic和final。先写出快照选择规则，再报告选中轮数。制作时选择第800轮：有效比例1.0、覆盖1/8、最大条件模式占比1.0。最终2200轮有效比例0，说明输出集中位置已经偏离数据团。两者不能合并成“最终有效比例100%的单模式结果”。

从stress_seed17_samples.npz读取800轮和2200轮数组，打印各坐标均值、标准差，并画在真实参考点上。它们都来自不同噪声经过训练后的G，不是后处理复制。再检查experiment.py，列出与基准不同的三个训练设置：G学习率、D学习率、每轮G更新次数。

```
import numpy as np
s = np.load('outputs/stress_seed17_samples.npz')
for step in ['800', '2200']:
    print(step, s[step].mean(0), s[step].std(0))
```

诊断任务D：为什么第800轮D真假均值都接近0.5，却不能宣布成功？请给出一个无需训练即可构造的反例，并指出必须补看哪两个分布指标。然后解释为什么这次压力实验只能支持“该配置发生了坍塌”，不能支持“G学习率是唯一原因”。

设计一个未执行的后续研究：保留基准参数，每次只改变一个因素，在同样种子和评价噪声下记录完整轨迹。写明计算预算如何比较，因为五次G更新的一轮不等于一次G更新的一轮。可以提议以总优化器更新次数或墙钟时间为额外横轴，但不要混淆两者。

请保留失败结果，不要只选最漂亮种子。若你修改配置后没有出现单模式快照，就如实报告未复现，并给出完整指标。随包检查点用于复核已发生的现象，重新训练用于探索你当前环境下的现象。科学报告允许不确定性，不允许为了匹配教材而伪造输出。

五 理论核对、Notebook与提交要求

推导任务E：固定G，对 $a \log d + b \log(1 - d)$ 求导，说明a、b同时为0时为什么判别器不唯一。加入真实来源先验 π 后重新推导，检验 $\pi=1/2$ 是否回到原公式。再用 $p=(1/2, 1/2)$ 、 $q=(1, 0)$ 计算JS和理想V，保留至少四位小数。

推理任务F：某同学说“理论证明GAN最小化JS，所以训练中生成器loss下降就是JS下降”。逐项指出问题：使用的是不是理想D、是否在做准确期望优化、G目标是否为经典极小极大、报告的loss是哪一方。给出一段不超过150字的正确表述。

Notebook由create_notebook.py生成，再由execute_notebook.py在新的IPython进程中顺序执行。它会重新运行全部训练，并将输出保存到notebook_outputs，避免覆盖教材的outputs。执行器使用无socket的in-process内核；这是代码顺序与输出验证，不是对Jupyter网页界面的测试。

```
python create_notebook.py
python execute_notebook.py experiment.ipynb
python build_pdf.py lecture.md
python build_pdf.py lab.md
python build_pdf.py answers.md
```

PDF构建需要requirements.txt中的Markdown、WeasyPrint和可用中文字体。若系统缺少Noto CJK，应按平台官方方式安装字体；也可先阅读Markdown。生成后使用PDF阅读器检查公式、图注、分页与代码行，不要只凭文件存在就认定可读。

最后提交一份实验记录：环境与测试结果；两步梯度边界检查；三种子指标；评价器负对照；压力运行诊断快照及最终状态；理想判别器推导；一个控制变量研究设计。将“实测结果”“教材已提供结果”和“计划中的实验”清楚区分。若报告包含新数值，应保留其生成命令、数据和参数文件，便于他人重新计算。

最低验收不是每个数字与教材逐位相同，而是过程可复查、论证不越界、失败未被掩盖。完成后，你能解释为何一个看似合理的loss和一张漂亮局部图，仍不足以说明生成分布学对了。